

Sistemas Distribuídos e Computação em Nuvem

Aula 1 — Abertura, diagnóstico e ambiente

C1 · Fundamentos e comunicação distribuída · 06/08/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — a trilha do semestre

C1 · Fundamentos e comunicação (Aulas 1–7 · 06/08 a 17/09)

Sockets, concorrência, gRPC, REST e **IA como serviço**.

C2 · Coordenação e consistência (Aulas 8–12 · 24/09 a 29/10)

Mensageria, relógios lógicos, **CAP**, **Raft**, resiliência.

C3 · Nuvem, implantação e segurança (Aulas 13–18 · 05/11 a 03/12)

Containers, nuvem, serverless, observabilidade, segurança/LGPD.

 Você está na **Aula 1**.

A disciplina em um slide

- **Ementa:** fundamentos e comunicação → coordenação e consistência → nuvem e implantação → segurança e LGPD.
- **Cinco módulos:** (1) fundamentos e comunicação · (2) serviços, APIs e **IA como serviço** · (3) coordenação, **CAP** e **Raft** · (4) nuvem, containers e implantação · (5) segurança, LGPD e tendências.
- **Fio condutor:** um **serviço de IA** que cresce de cliente-servidor a **plataforma cloud-native** — a IA é **caixa-preta** (você chama API/biblioteca, **nunca treina modelo**).
- **Como serão as aulas:** 90 min **práticos** — o laboratório **começa na aula e termina em casa**; Git e nuvem desde o primeiro dia.

 Você aprende sistemas distribuídos **construindo**, por partes, um serviço de IA cloud-native.

Como você será avaliado

- Três verificações — C1, C2 e C3. Cada uma vale até 10,0 = duas avaliações de 5,0.
- A1 — Prova escrita estilo ENADE (5,0). Datas: C1 17/09 · C2 29/10 · C3 26/11.
- A2 — Trabalho prático sem apresentação (5,0): código + README entregue no repositório.
- Rubrica do trabalho: arquitetura 1,5 · comunicação 1,5 · resiliência 1,0 · execução reproduzível 1,0 — a sofisticação da IA não pontua.

💡 Nota Final = $(C1 + C2 + C3) / 3$. Em cada ciclo: uma prova (teoria ENADE) + um trabalho prático.

Objetivos desta aula

Ao final, você será capaz de:

1. **Explicar** o que é um sistema distribuído e por que quase todo software hoje é um.
2. **Reconhecer** o desafio que só existe quando há rede no meio: a **falha parcial**.
3. **Deixar o ambiente pronto**: Python, VS Code, Git e GitHub (VM a partir de amanhã).

Conceito 1/3 — O que é um sistema distribuído

- Vários **computadores independentes** que cooperam pela rede e se apresentam como **um sistema único**.
- Você usa dezenas por dia: **WhatsApp, Netflix, app do banco, Uber, ChatGPT**. Ao ver o saldo, dezenas de máquinas cooperaram — você enxerga só o resultado.
- A cooperação traz **3 ganhos**: **escalabilidade** (mais máquinas, não uma maior), **disponibilidade** (segue de pé se uma cai) e **proximidade** do usuário (servidores por região).
- O preço: existe **uma rede no meio** — lenta, imprevisível e que **falha** (e não falha de forma limpa).



Sistema distribuído = várias máquinas cooperando que **parecem uma só**.

Duas ideias que acompanham o curso

Latência

- Tempo para uma mensagem ir de um ponto a outro da rede.
- Memória local: **nanossegundos**; rede: **milissegundos** — e **variável**.
- Muitas decisões de projeto existem para **esconder/reduzir** a latência.

Transparência

- Esforço de **esconder** a distribuição do usuário.
- De **acesso** (usar remoto como local), de **localização** (não saber onde está), de **replicação** (não perceber as cópias).
- Mais transparência por fora = mais trabalho por dentro.

Conceito 2/3 — Falha parcial (o problema central)

- Num sistema **centralizado**, a falha é **total**: ou tudo funciona, ou tudo para — e fica **evidente**.
- Num sistema **distribuído** surge a **falha parcial**: uma parte para enquanto o resto continua.
- Pior é a **ambiguidade**: se **A** pede a **B** e não recebe resposta, **A não sabe** se B **caiu**, está **lento** ou se a **resposta se perdeu**. As três são **indistinguíveis** para A.

⚠ Guarde esta ideia: relógios lógicos, CAP, consenso e resiliência existem **por causa** da falha parcial. Aprender SD é aprender a projetar **contando com ela**.

Conceito 3/3 — O que você vai construir

- Uma **plataforma de IA como serviço** que nasce como um par **cliente-servidor** e, aula após aula, ganha **interfaces modernas, fila, microsserviços, containers** e vai para a **nuvem**.
- A **IA é a tarefa** que o sistema executa: você **chama um modelo pronto, nunca o treina** — sem matemática de aprendizado de máquina.
- Tudo **versionado no Git** desde o 1º dia: é como equipes reais colaboram — e como você **provará** o que construiu.

 **C1.A2 — Serviço de Inferência Distribuído** começa hoje: preparar o ambiente e criar o repositório onde ele vai crescer.

Como usar este material (leia primeiro)

Onde está o código: todo exemplo citado nos slides (`servidor_eco.py` , esqueletos...) está no **repositório do curso**, em `aulas/exemplos/aulaNN/` .

```
git clone https://github.com/howardroatti/sistemas-distribuidos-faesa.git  
# sem Git? No GitHub: botão verde Code → Download ZIP
```

- **Dois terminais** (vários labs pedem): no VS Code, menu **Terminal** → **Split Terminal**.
- **Ativar o venv barrou?** Se o PowerShell disser "*scripts is disabled*", rode **uma vez**: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responda **S**).

💡 Pouca prática com Python/terminal/HTTP? Faça o **roteiro de nivelamento** (`nivelamento.md` no repositório) antes de começar.

Ambiente — hoje, sem a VM ainda



A **máquina virtual** da disciplina fica **disponível amanhã**. Hoje usamos o **Windows nativo** do laboratório — que já tem **Python** e **VS Code**. É o suficiente para tudo de hoje.

No PowerShell, confirme que está tudo à mão:

```
python --version      # deve mostrar Python 3.x (se não achar, tente: py --version)
code --version         # VS Code instalado
git --version          # se aparecer versão, ótimo; se não, use o Plano B (próximo slide)
```

💡 Os laboratórios de hoje e da próxima aula são **Python puro** — **não** precisam de Docker nem da VM. Docker entra só nos trabalhos, já na VM.

Ambiente — seu repositório `sd-2026-2`

Caminho A — com Git (recomendado)

```
cd $HOME\Documents
git init sd-2026-2
cd sd-2026-2
# ...crie seus arquivos...
git add .
git commit -m "primeiro commit"
# crie o repo no GitHub e:
git remote add origin URL_DO_SEU_REPO
git push -u origin main
```

Caminho B — sem Git hoje (plano B)

1. Entre no github.com e clique em **New repository**.
2. Nome: `sd-2026-2` → **Create**.
3. Guarde seus arquivos numa pasta local hoje.
4. **Amanhã, na VM**, faça o `commit / push` — ou use **Add file → Upload** pelo site.

💡 O objetivo de hoje é **ter o repositório criado**. O push pode ser concluído na VM amanhã.

Kit da C1.A2 — clone e experimente (hoje)

O trabalho da **C1** já tem um **kit de partida** público. A **rota síncrona roda offline, sem Docker** — dá para provar **hoje, no Windows**:

```
git clone https://github.com/howardroatti/sd-2026-2-kit-c1a2.git
cd sd-2026-2-kit-c1a2
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # ativa o ambiente (PowerShell)
pip install -r requirements.txt
uvicorn app.api_rest:app --reload --port 8000 # abra http://localhost:8000/docs
# em OUTRO terminal (com o .venv ativo):
python exemplos/cliente_rest.py "o atendimento foi otimo"
```

💡 O "modelo de IA" é um classificador de sentimento **scikit-learn** que treina sozinho na 1ª execução — **100% offline**. A **fila (Docker/Redis)**, o **worker** e o **gRPC** entram na **VM (amanhã)**; o desafio completo abre na **Aula 6**.

Atividade em sala

Diagnóstico (sem nota) — à prova de LLM

Diagnóstico — como funciona (35 min)

Sem consulta e sem celular · em papel · não vale nota (serve para eu calibrar o ritmo das aulas).

Parte A (10 min) — Desenho

Desenhe à mão o **caminho de um "oi"** no chat, do seu celular ao do colega. Nomeie as "caixas" que imaginar.

Parte B (10 min) — Predição

B1 (*antes*): o que acontece se o servidor cair no meio de uma requisição?

B2/B3 (*depois da demo*): o que **de fato** ocorreu e por quê.

Parte C (15 min) — Quiz sobre o log

Você responde **5 questões** olhando **apenas** para o **log projetado** da demonstração ao vivo.



Não há resposta "certa" no desenho — é o seu **modelo mental de hoje**. No fim do semestre você o refaz.

Demonstração ao vivo (professor)

Dois terminais no projetor, com os exemplos `exemplos/aula01/servidor_eco.py` e `cliente_eco.py`:

1. **Terminal 1:** `python servidor_eco.py` — ele fica **ouvindo** na porta 5000.
2. **Terminal 2:** `python cliente_eco.py` — envia `"oi"` e **aguarda** a resposta.
3. O servidor **demora de propósito** (simula processamento). Enquanto o cliente espera...
4. **No Terminal 1, aperte `Ctrl + C`** — o servidor **cai** no meio da requisição.
5. Observe o **erro do cliente** e **projete o log** — é o artefato da **Parte C**.

⚠ Frase-chave ao fechar: **"o cliente nunca sabe o que aconteceu do outro lado."**

💡 Os dois arquivos estão prontos no repositório (`exemplos/aula01/`) — rodam no **Windows**, sem VM.

Diagnóstico — gabarito da Parte C

O log da demo mostra a **falha parcial**. As respostas apontam para o mesmo lugar:

- **C1 = B** — não houve **aviso**; a conexão só parou.
- **C2 = B** — o cliente **esperou alguns segundos** até o tempo-limite.
- **C3 = B** — "caiu" × "lento" são **indistinguíveis** (*a pergunta mais importante*).
- **C4 = B** — reenviar pode **executar duas vezes** → semente de **idempotência** (Aula 11).
- **C5 = B** — faltou **tempo-limite + nova tentativa** → **resiliência** (Aula 11).

Atividade para casa

1. **Confirme** o ambiente: `python --version` e (se houver) `git --version`.
2. **Crie** o repositório `sd-2026-2` no GitHub (Caminho A ou B do slide de ambiente).
3. **Rode** o exemplo `servidor_eco.py` / `cliente_eco.py` no seu Windows (dois terminais) e observe o erro ao derrubar o servidor.
4. **Espie o futuro:** clone o **kit da C1.A2** e rode a **rota síncrona** (passos no slide "*Kit da C1.A2*").
5. **Opcional:** se tem pouca prática com Python/HTTP, faça o **roteiro de nivelamento** (`nivelamento.md` no repositório).

📌 **Entregar até a próxima aula:** link do repositório `sd-2026-2` criado (o 1º commit pode ser concluído amanhã na VM).

◆ Foco ENADE

O que costuma cair:

- Conceito e **caracterização** de sistemas distribuídos.
- Vantagens e desafios: **escalabilidade, disponibilidade e falha parcial**.
- Diferença **centralizado × distribuído**.
- Tipos de **transparência** (acesso, localização, replicação).

Termos-chave: Sistema distribuído · Falha parcial · Transparência · Escalabilidade · Latência

💡 A falha parcial aparece **disfarçada em estudos de caso**; a latência costuma justificar decisões de arquitetura.

Questão de autoavaliação (estilo ENADE)

Um serviço **A** envia uma requisição ao serviço **B** e **não recebe resposta** no tempo esperado. Assinale a alternativa correta.

- A) A pode concluir **com certeza** que B falhou.
- B) A pode concluir que a requisição **certamente não** foi processada.
- C) A **não distingue**, só pela ausência de resposta, entre B ter caído, estar lento ou a resposta ter se perdido.
- D) A deve **encerrar todo o sistema**, pois é falha total.
- E) A situação **não ocorre** com TCP.

Resolução — alternativa C

- É a definição de **falha parcial**: a ausência de resposta é **ambígua**.
- O **TCP não elimina** o problema — B pode cair **depois** de receber o pedido.
- As demais afirmam **certezas** que A **não possui**.

 Repare como o mesmo cenário da demo de hoje vira questão de prova.

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 1 — Caracterização de SD.
- **Tanenbaum & Van Steen**, *Distributed Systems*, 4ª ed., cap. 1 (PDF grátis dos autores).
- **Guarde o desenho de hoje** — você vai refazê-lo no fim do semestre.

Glossário

- **Sistema distribuído**: máquinas independentes que se apresentam como uma só.
- **Falha parcial**: parte falha, resto continua.
- **Latência**: tempo de ida de uma mensagem na rede.
- **Transparência**: esconder a complexidade da distribuição.
- **Repositório**: pasta versionada pelo Git.

Até a próxima aula

Entregue o repositório `sd-2026-2`. A Aula 2 começa revendo isto.

≡ Índice

todas as aulas

Próxima aula →

Aula 2 · Sockets (TCP/UDP)